

OOPs with C++



Programming language

A **programming language** is a formal computer **language** designed to communicate instructions to a machine, particularly a computer. **Programming languages** can be used to create programs to control the behaviour of a machine or to express algorithms.

Machine oriented languages or low level language

A low-level language is a programming language that provides little or no abstraction of programming concepts, and is very close to writing actual machine instructions. Two good examples of low-level languages are assembly and machine code.

Problem Oriented language or High level language

A high-level language is a computer programming language that isn't limited by the computer, designed for a specific job, and is easier to understand. It is more like human language and less like machine language. However, for a computer to understand and run a program created with a high-level language, it must be compiled into machine language. Example: BASIC, C, C++, Cobol, FORTRAN, Java, Pascal, Perl, PHP, Python,

C++ and Object Oriented Programming

Object Oriented programming is a programming style that is associated with the concept of Class, Objects and various other concepts revolving around these two, like Inheritance, Polymorphism, Abstraction, Encapsulation etc.

Class:

- It is similar to structures in C language. Class can also be defined as user defined data type but it also contains functions in it. So, class is basically a blueprint for object. It declare & defines what data variables the object will have and what operations can be performed on the class's object.

Objects

- Objects are the basic unit of OOP. They are instances of class, which have data members and uses various member functions to perform tasks.

Abstraction: Abstraction refers to showing only the essential features of the application and hiding the details. In C++, classes provide methods to the outside world to access & use the data variables, but the variables are hidden from direct access. This can be done access specifiers.

Encapsulation : It can also be said data binding. Encapsulation is all about binding the data variables and functions together in class.

Inheritance : Inheritance is a way to reuse once written code again and again. The class which is inherited is called base class & the class which inherits is called derived class. So when, a derived class inherits a base class, the derived class can use all the functions which are defined in base class, hence making code reusable.

Polymorphism : It is a feature, which lets us create functions with same name but different arguments, which will perform differently. That is function with same name, functioning in different way. Or, it also allows us to redefine a function to provide its new definition. You will learn how to do this in details soon in coming lessons.

Class in C++

- A class is way to bind the data and its associated function together. It allows data to be hidden if necessary, form external use. When define a class we are creating a new abstract data type that can treated like any other data type.

Generally a class specification has two part:

- Class declaration
- Class Definition
- The class declaration describes the & scope of its members.
- The class function describes how the class functions are implemented.
- General form of a class declaration

```
class <class name>
{
    visibility mode:
    variable declaration;
    function declaration() { }
}
```

Visibility Mode

- It is used to specify the three levels of access for hiding data and functions to the class.
- Private:
 - In private section a member data can only be accessed by the member of function & friend function of that class. The private data members are not accessible to outside of the class even through the object of that class.
- Protected:
 - The behavior of protected data members are the same as private data members but these can be inherited to derived class .
- Public:

the member of this section can be accessed by any function outside of the class. The public implementation operations are also called member function or method or interface to outside of the class. Any function can send message to object of this class by these interface functions.

Object in C++

Creating Object: an object is instance of a class and can be created by using class name.

```
<class name> <object name>
```

```
Student s1;
```

Object can be created when a class is defined by placing their names immediately after the closing braces (}).

```
Student
```

```
{
```

```
.....
```

```
.....
```

```
}s1,s2;
```

Assessing Class Member: The private data of a class can be accessed only through the member functions of that class.

```
object_name.data_member_name;
```

```
object_name.function_name(argument list);
```

Ex: obj.ShowData();

Member Function in C++

- Member function can be defined into two place:
 - Inside class
 - Outside Class
- **Inside Class:** when a member function is defined in inside a class it is treated as an inline function. If a function very small(not contain switch or loop) then it should be defined inside the class.
- **Outside Class:** When a function has large code (contain switch case or loop) then it should be defined outside of the class. The prototype of such function must be provided in the class definition.

The operator (::) known as scope resolution operator is used to associated member function to there corresponding class.

< return type > < class name > :: < function name>(parameter list)

{

function body

.....

}

Nesting of Member Function

- A member function can be called by using its name inside an other member function of the same class. This is known as Nesting member function.

```
class SquareNumber
{
    int number;
    public:
    long square()
    {
        number=number*number;
        return (number);
    }
    void FindSquare(int x)
    {
        number=x;
        cout<<"Square of the number:"<<square();
    }
};
```

Function Overloading

- If any class have multiple functions with same names but different parameters then they are said to be overloaded. Function overloading allows you to use the same name for different functions, to perform, either same or different functions in the same class.

Ways to overload a function

- By changing number of Arguments.
- By having different types of argument.

```
int sum (int x, int y) {  
    cout << x+y; }
```

```
int sum(int x, int y, int z) {  
    cout << x+y+z; }
```

```
double sum(double x,double y) {  
    cout << x+y; }
```

Function with Default Arguments

- A default argument is a value provided in function declaration that is automatically assigned by the compiler if caller of the function doesn't provide a value for the argument with default value.
- Default arguments are overwritten when calling function provides values for them.
- During calling of function, arguments from calling function to called function are copied from left to right.

Example:

```
int sum(int x, int y, int z=0, int w=0)
{
    return (x + y + z + w);
}
```

Constructor

- A constructor is a special member function whose task is to initialize the data member of its class. It is special because its name is the same as the class name.

The constructor have some special characteristics:

- The constructor is invoked automatically when the object of that class is created.
- It do not have any return type ,not even void.
- They should be declared in public section.
- They cannot be inherit in derived class.
- Like other C++ function they can have default argument.
- Constructor cannot b e virtual.

Types of Constructor

- Default Constructor
- Parameterized Constructor
- Copy Constructor

Default Constructor

- A constructor without any arguments or with default value for every argument, is said to be **default constructor**.

```
class Bank
{
    int ac_no;
    double amount;
public:
    Bank()
    {
        ac_no=101;
        amount=5000;
    }
}
```

Parameterized Constructors

- It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function.
- Example:

```
class Bank
{
    int ac_no;
    double amount;
    public:
    Bank(int a,double amt)
    {
        ac_no=a;
        amount=amt;
    }
}
```

Copy Constructor

- A copy constructor is a member function which initializes an object using another object of the same class.

```
class Bank
{
    int ac_no;
    double amount;
public:
    Bank(Bank b)
    {
        ac_no=b. ac_no;
        amount=b. amountt;
    }
}
```

